



Internship Report

Development of a Mobile Application and Raspberry Pi-Based Control System for a Robotic Tennis Ball Machine



**University of
Sheffield**

/AMRC
Training Centre

Supervisor: Doctor Lin Cao, Lecturer, University of Sheffield

Internship performed at The University of Sheffield from 04/06/26 to 28/07/26

Andy Mannikum

andymannikum76@gmail.com

<https://github.com/AndyM764/AndyM764.github.io>

Acknowledgement

I would like to express my sincere gratitude to Dr Lin Cao for providing me with the opportunity to undertake this project at the University of Sheffield. This experience allowed me to develop my technical and practical skills while gaining valuable knowledge through hands-on work and exposure to a research environment. I am particularly grateful for his guidance, support, and valuable advice throughout the project.

I would also like to thank Zheyuan Bi, a PhD student in the laboratory, for his assistance and guidance during my time there. His help in locating the necessary tools, components, and equipment, as well as his valuable advice throughout the project, was greatly appreciated and contributed to the successful completion of my work.

Finally, I am grateful to everyone who contributed to making this experience a valuable and rewarding learning opportunity.

Table of Contents

ACKNOWLEDGEMENT	2
1. INTRODUCTION	4
2. SYSTEM OVERVIEW	4
3 OVERALL SYSTEM DESIGN	6
3.1 HARDWARE ARCHITECTURE	6
<i>Raspberry Pi GPIO and Motor Driver Connections.....</i>	<i>6</i>
3.2 SOFTWARE ARCHITECTURE	7
3.3 COMMUNICATION FLOW	8
3.4 NETWORK ARCHITECTURE	10
4. DESIGN DECISIONS	11
4.1 DESIGN METHODOLOGY AND DEVELOPMENT APPROACH	11
4.2 SELECTION OF RASPBERRY PI 5	12
4.3 SELECTION OF A NATIVE IOS APPLICATION	12
4.4 SELECTION OF HTTP COMMUNICATION	13
4.5 SELECTION OF MJPEG STREAMING	13
4.6 SELECTION OF A STANDALONE HOTSPOT	13
4.7 AUTOMATIC SYSTEM STARTUP	14
5. IMPLEMENTATION	14
5.1 RASPBERRY PI SOFTWARE	14
5.2 IOS APPLICATION	16
5.3 LIVE VIDEO STREAMING	18
5.4 VIDEO RECORDING	20
5.5 VIDEO DOWNLOAD	23
5.6 PARAMETER TRANSMISSION	25
5.7 WIRELESS NETWORK CONFIGURATION	28
<i>SSH Access During Development</i>	<i>29</i>
5.8 SYSTEM INTEGRATION	29
6 TESTING AND EVALUATION	31
7 CHALLENGES AND LIMITATIONS	31
7.1 MOBILE APPLICATION DEVELOPMENT	31
7.2 STEPPER MOTOR IDENTIFICATION AND CONTROL	32
7.3 LIMITED HARDWARE DIAGNOSTIC EQUIPMENT	33
7.4 PROJECT TIME CONSTRAINTS	34
8 FUTURE IMPROVEMENTS	34
9 CONCLUSION	35

1. Introduction

Tennis ball machines are widely used during training to provide players with consistent and repeatable ball delivery. Modern training systems increasingly require wireless control and monitoring to improve usability, allowing coaches and players to adjust machine settings without physically interacting with the equipment.

The aim of this project is to develop a wireless control and monitoring system for a smart tennis ball machine using a Raspberry Pi 5 and a native iOS application. The system enables an operator to view a live camera feed, record training sessions, download recorded videos, and remotely configure the operating parameters of the machine from a mobile phone.

In addition to providing these functions, the system has been designed to operate independently of external network infrastructure, making it suitable for use on tennis courts. Particular emphasis has been placed on developing a reliable and modular software architecture that can be extended to support additional functionality, including motor control and future intelligent features(Tracknet).

This report describes the requirements, design, implementation and testing of the developed system. It also discusses the engineering decisions made throughout the project, the challenges encountered during development, and possible improvements for future work.

2. System Overview

The developed system consists of two main components: a Raspberry Pi 5 installed on the tennis ball machine and a native iOS application running on an iPhone. The Raspberry Pi acts as the central controller, performing image acquisition, video recording, wireless communication and processing control commands received from the mobile application. The iOS application provides the graphical user interface through which the operator monitors and controls the system.

Communication between the two devices is established over a dedicated wireless connection. The mobile application exchanges HTTP requests with the Raspberry Pi to perform operations such as starting and stopping video recording, downloading recorded footage and transmitting machine operating parameters. This architecture separates the user interface from the embedded processing tasks, simplifying future development and maintenance.

The system has been designed to operate as a self-contained platform. During startup, the Raspberry Pi automatically initialises the required software services and creates its own wireless hotspot, allowing the mobile application to connect directly without relying on external network infrastructure. This improves portability and enables the system to be deployed in environments where internet access may not be available.

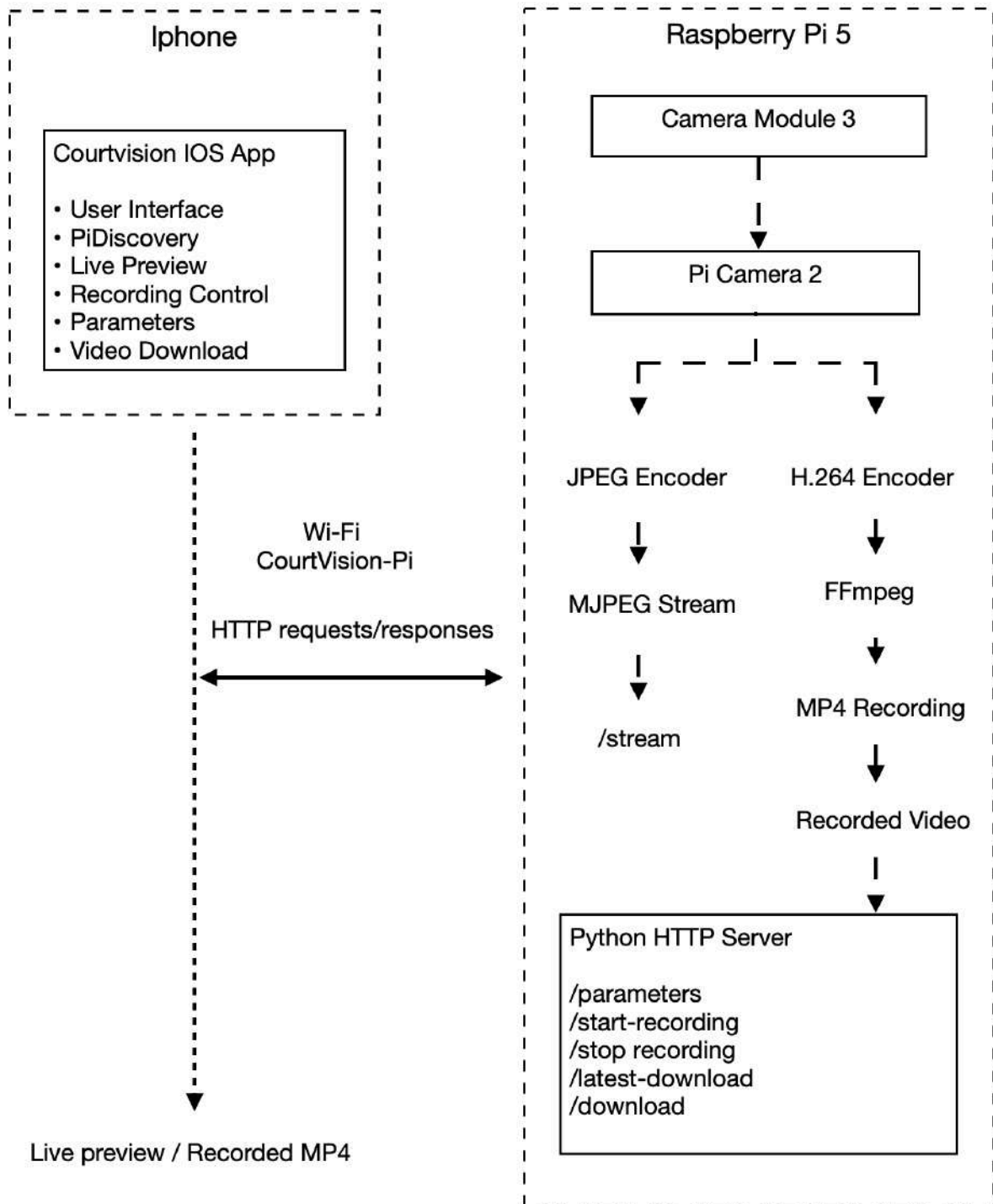


Figure 2.1 presents the overall architecture of the developed system.

3 Overall System Design

3.1 Hardware Architecture

The hardware platform is centred around the Raspberry Pi 5, selected for its processing capability and support for peripheral devices required by the project. A Raspberry Pi Camera Module 3 is connected directly to the CSI interface and provides live video for monitoring and recording. The Raspberry Pi also provides GPIO interfaces which are used for communication with the stepper motor driver responsible for controlling the tennis ball launching mechanism.

The operator interacts with the system using an iPhone running the developed SwiftUI application. Rather than communicating through an external router, the mobile device connects directly to the wireless hotspot generated by the Raspberry Pi, allowing the complete system to function as a standalone embedded platform.

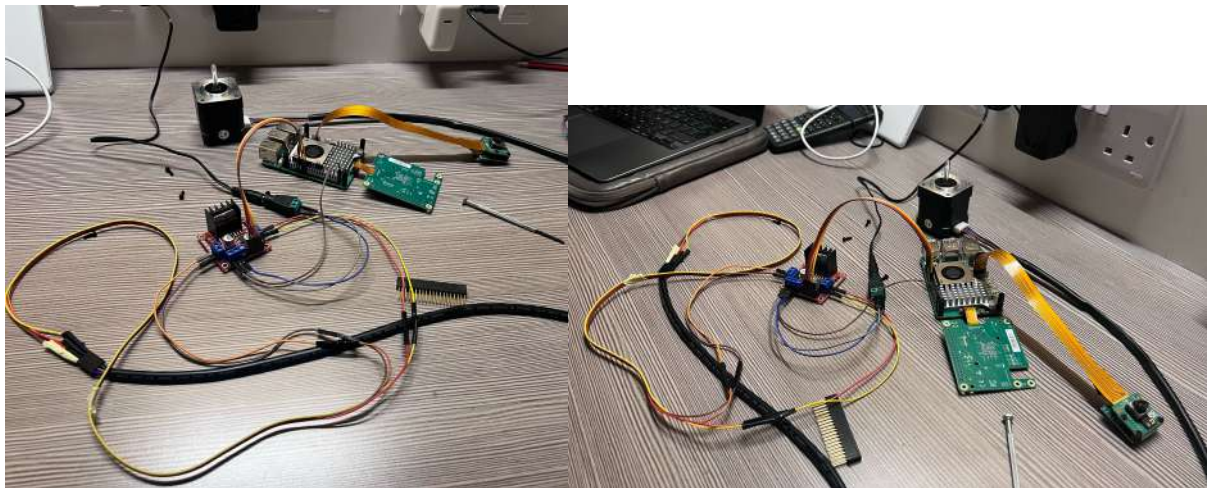


Figure 2.2 illustrates the hardware components and their interconnections.

Raspberry Pi GPIO and Motor Driver Connections

Figure 2.3 shows the electrical connections used during the stepper-motor investigation. The Raspberry Pi 5 provided the control signals through its GPIO pins, which were connected to the input pins of the L298N motor driver. The driver then supplied the required signals to the two motor coils through its output terminals.

The motor driver was powered from an external 12 V, 2 A supply, while the Raspberry Pi was used to provide the control signals. The GPIO-to-driver connections used in the implementation were GPIO17, GPIO4, GPIO27 and GPIO14. These connections corresponded to physical pins 11, 7, 13 and 8 on the Raspberry Pi header, respectively.

The stepper motor used four active wires connected to the L298N output terminals. The circuit was assembled and tested as part of the motor-control investigation; however, reliable motor rotation was not achieved and the motor subsystem was therefore not integrated into the final CourtVision application.

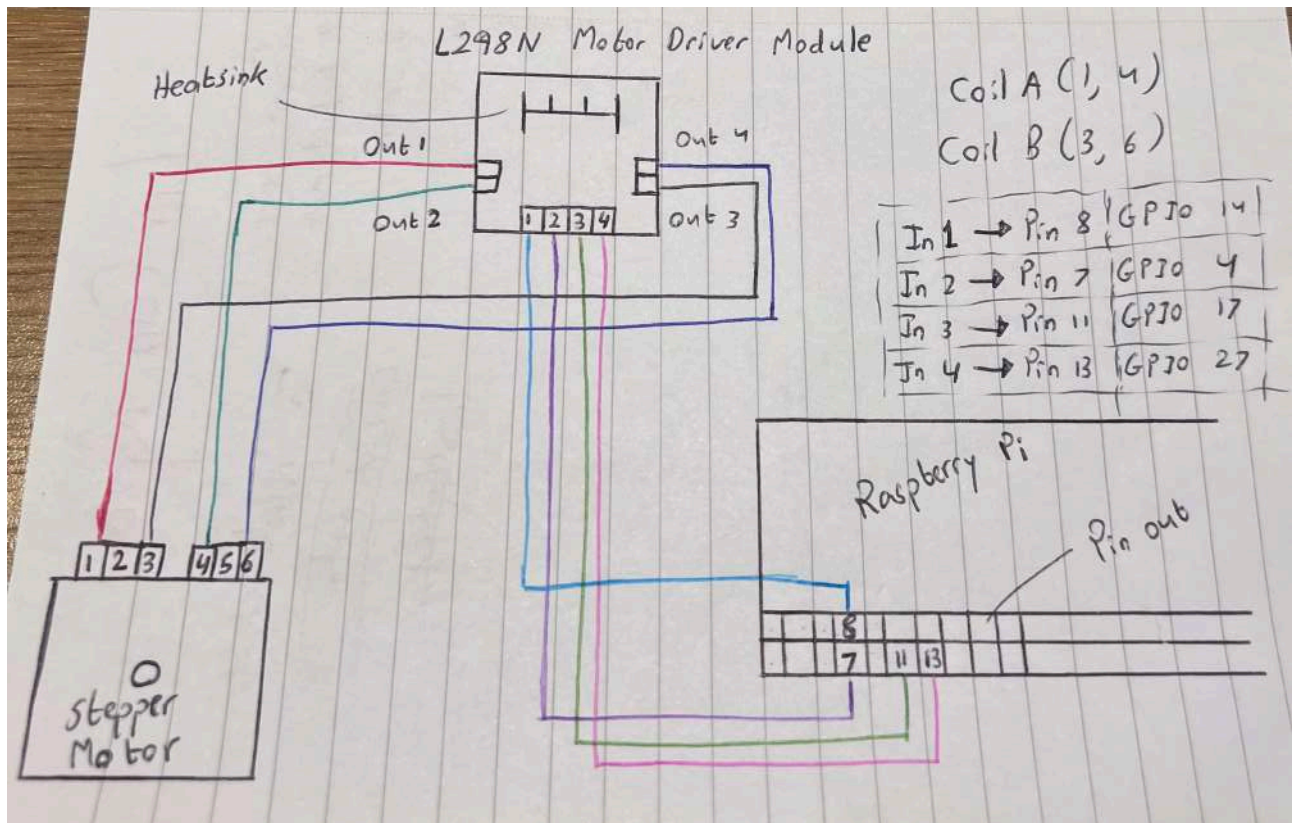


Figure 2.3 Circuit diagram showing the Raspberry Pi GPIO, L298N motor driver and stepper motor connections.

3.2 Software Architecture

The software architecture is divided into two independent layers.

The first layer consists of the iOS application, which provides the graphical interface and manages user interaction. Functions implemented within the application include displaying the live camera stream, controlling video recording, downloading recorded videos and transmitting operating parameters to the Raspberry Pi.

The second layer consists of software executing on the Raspberry Pi. A lightweight Python HTTP server manages communication with the mobile application while interfacing with the camera and other hardware components. Individual HTTP endpoints are responsible for handling video streaming, recording requests, parameter updates and file transfers. Separating these functions into dedicated services improves modularity and simplifies debugging during development.

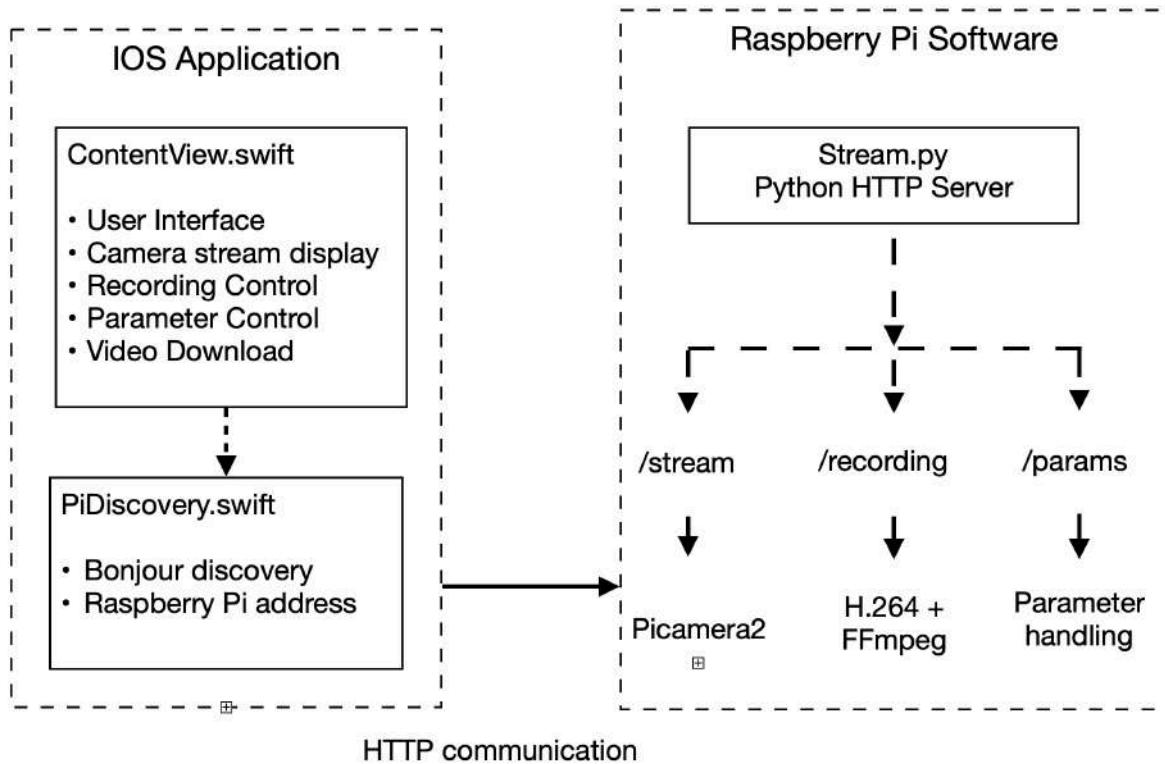


Figure 2.4 illustrates the relationship between the software modules implemented on both the iPhone and Raspberry Pi.

3.3 Communication Flow

Communication between the iOS application and the Raspberry Pi is achieved using HTTP requests over a wireless network. The mobile application acts as the client, while the Raspberry Pi hosts a lightweight Python HTTP server that processes incoming requests and returns the appropriate responses.

Each function within the application communicates with a dedicated HTTP endpoint. For example, requests are sent to start and stop video recording, download the latest recorded video, and transmit the operating parameters entered by the user. Machine parameters, including ball speed, launch frequency and launch angle, are encoded using JSON before being transmitted to the Raspberry Pi, where they are processed for use by the control software.

The live camera preview follows a different communication path. Instead of repeatedly requesting individual images, the application connects to a continuous MJPEG video stream generated by the Raspberry Pi. This allows the operator to monitor the machine in real time while using the same wireless connection for control commands.

Separating streaming, recording, downloading and parameter transmission into independent HTTP endpoints simplified the software architecture and made individual functions easier to develop, test and maintain.

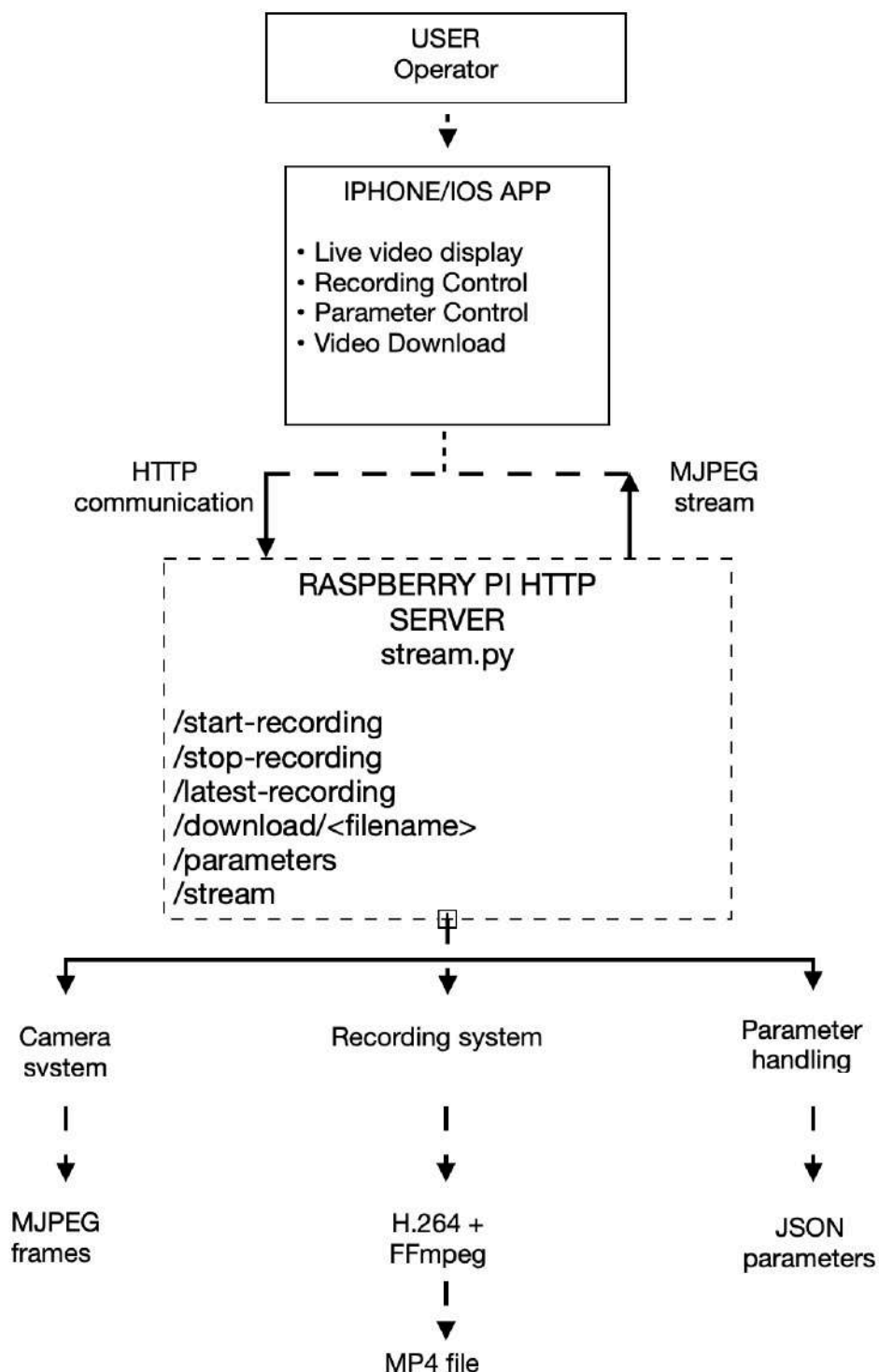


Figure 2.5 – Communication flow between the iOS application and Raspberry Pi.

3.4 Network Architecture

The system was designed to operate without dependence on external network infrastructure. Rather than requiring an existing wireless network, the Raspberry Pi is configured to automatically create its own Wi-Fi hotspot during system startup. The phone connects directly to this hotspot, establishing communication with the Raspberry Pi without requiring internet access or a third-party router.

This approach improves the portability of the system, allowing the tennis ball machine to be deployed in locations such as outdoor tennis courts where network availability cannot be guaranteed. Once connected to the hotspot, all communication between the mobile application and the Raspberry Pi remains within the local network, including live video streaming, recording control, parameter transmission and video downloads.

To improve usability, the hotspot and streaming server are automatically started during the Raspberry Pi boot process. As a result, the operator only needs to power the system and connect the mobile device to the Raspberry Pi hotspot before launching the application.

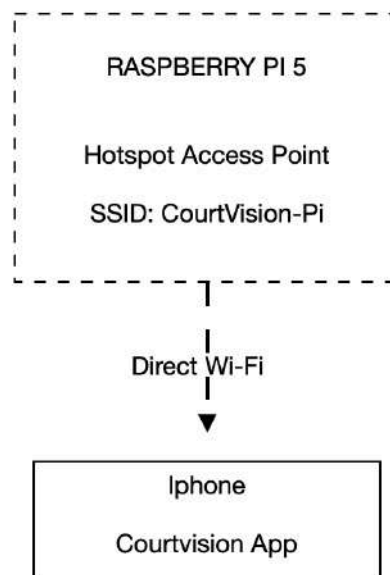


Figure 2.6 – Network architecture showing direct communication between the iPhone and Raspberry Pi hotspot.

- Local network only — no external router or Internet connection required

4. Design Decisions

4.1 Design Methodology and Development Approach

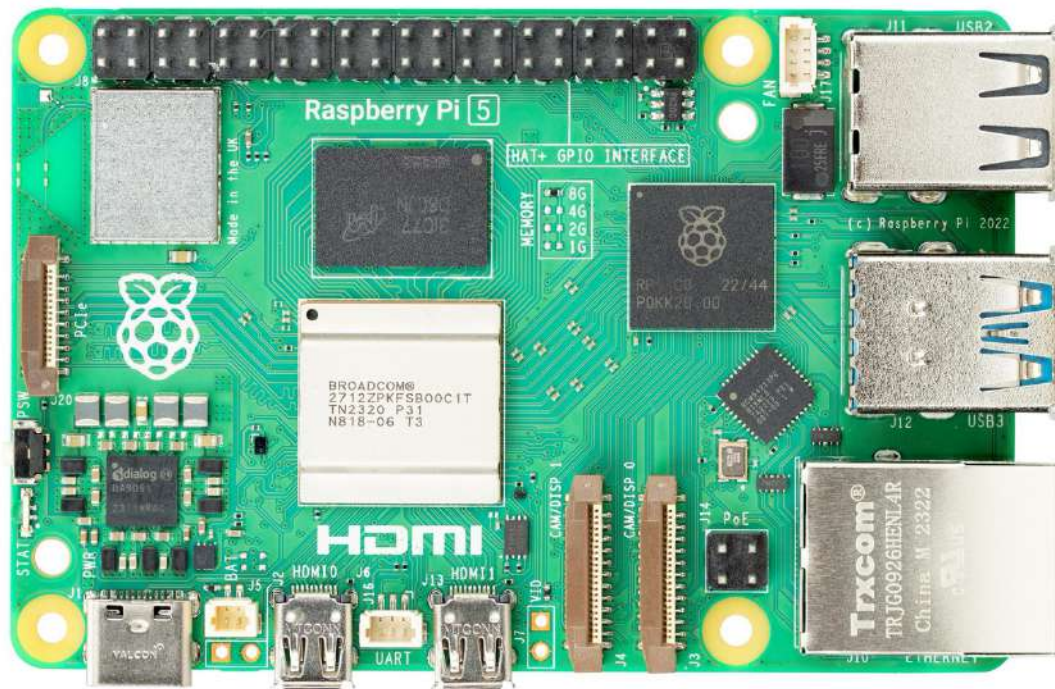
The system was developed using an iterative and modular approach. Rather than developing the complete system as a single unit, the mobile application, Raspberry Pi software, camera system and motor-control hardware were initially treated as separate subsystems. Each subsystem was developed and tested independently before being progressively integrated into the complete system. This approach was adopted to simplify debugging and allow problems within individual components to be identified without affecting the remaining system.

Development was carried out through repeated cycles of implementation, testing and refinement. Software functionality was first verified using the individual components and communication interfaces before integration with the complete system. This allowed the camera, wireless communication and mobile application functionality to be established before attempting full hardware integration.

The motor-control subsystem was investigated separately from the completed camera and communication systems. Different GPIO configurations, motor-coil connections and stepping sequences were tested to determine whether reliable rotation could be achieved using the available stepper motor and L298N driver. Although reliable motor rotation was not achieved within the project timeframe, separating this investigation from the other subsystems allowed the functional parts of the system to be completed and provided a clear basis for identifying the remaining hardware limitations.

The specific technologies and design approaches selected during this process, including the Raspberry Pi 5, native iOS application, HTTP communication, MJPEG streaming and standalone wireless hotspot, are discussed in the following sections.

4.2 Selection of Raspberry Pi 5



The project required a computing platform capable of simultaneously handling camera streaming, video recording, wireless communication and future motor control. Several embedded platforms could potentially satisfy parts of these requirements; however, the Raspberry Pi 5 was selected because it provides sufficient processing performance while also offering direct access to hardware interfaces through its GPIO pins.

An Arduino-based solution was not considered suitable due to its limited processing capability and inability to support real-time video streaming or video recording. Although an ESP32 provides integrated wireless communication, its available processing resources would significantly limit future expansion of the system.

The Raspberry Pi also benefits from mature software support, including the Picamera2 library for camera integration and a Linux operating system capable of running multiple software services concurrently. This allowed both the mobile communication server and camera processing software to execute simultaneously without requiring additional hardware.

4.3 Selection of a Native iOS Application

A native iOS application was selected to provide the user interface for the tennis ball machine. Since the intended control device was an iPhone, developing the application using SwiftUI allowed direct access to native device functionality while providing a responsive and consistent user experience.

Alternative approaches, such as a browser-based web interface or cross-platform frameworks, were considered less suitable for this project. A native application simplified integration with

device features including the Photo Library, which was required for storing downloaded recordings. It also provided a cleaner interface for future expansion as additional machine functions are introduced.(expo go was used before but it had difficulties handling multiple requests as same time, could not live stream and record simultaneously)

4.4 Selection of HTTP Communication

The system required a reliable method of exchanging commands and data between the mobile application and the Raspberry Pi. Since wireless networking was already required for live video streaming, HTTP was selected as the communication protocol for all control functions.

Using a single protocol simplified the software architecture by allowing recording requests, parameter transmission and video downloads to be handled through dedicated HTTP endpoints. In addition, HTTP requests could be tested independently during development using standard tools such as a web browser and curl, simplifying debugging throughout implementation.

Machine operating parameters are transmitted using JSON, providing a lightweight and easily extensible data format. Should additional machine settings be required in future, further parameters can be incorporated without changing the overall communication architecture.

4.5 Selection of MJPEG Streaming

Several methods exist for transmitting live video over a network, including WebRTC, RTSP and Motion JPEG (MJPEG). Although technologies such as WebRTC provide lower latency, they require considerably greater implementation complexity, including signalling and additional media handling.

For the objectives of this project, MJPEG provided an appropriate balance between performance and implementation effort. The stream could be generated directly by the Raspberry Pi and displayed within the mobile application without introducing additional streaming infrastructure. This reduced software complexity while still providing sufficient image quality and responsiveness for monitoring the tennis ball machine during operation.

4.6 Selection of a Standalone Hotspot

Early development was carried out using an existing wireless network(flat wifi which was booted on sd card). While this approach was adequate during testing, it limited system portability because operation depended on the availability of external Wi-Fi infrastructure.

To remove this dependency, the Raspberry Pi was configured to automatically generate its own wireless hotspot. The mobile application therefore communicates directly with the Raspberry Pi, allowing the complete system to operate independently of internet connectivity

or third-party networking equipment. This approach better reflects the intended deployment environment, where the machine may be used on outdoor tennis courts or training facilities without reliable network access.

4.7 Automatic System Startup

The completed system was intended to behave as a dedicated embedded device rather than a general-purpose computer. Requiring manual execution of the streaming server after every power cycle would reduce usability and increase setup time.

To address this, the streaming application was configured as a systemd service that automatically starts during Raspberry Pi boot. Combined with the automatic hotspot configuration, this allows the system to become operational shortly after power is applied, requiring only that the operator connect the mobile device to the Raspberry Pi hotspot before launching the application.

5. Implementation

5.1 Raspberry Pi Software

The Raspberry Pi 5 acts as the central embedded computer within the developed system. It is responsible for camera management, wireless communication with the iOS application, video recording and execution of system commands. The Raspberry Pi was operated in a **headless Linux configuration**, meaning that no graphical desktop environment was required during normal operation. All required functionality was implemented within a single Python application, simplifying deployment and reducing the number of software services required on the device.

Python was selected for the Raspberry Pi implementation because of its compatibility with the Linux environment and the availability of libraries for camera control and video processing. The application uses the **Picamera2** library to interface with the Raspberry Pi Camera Module 3 and Python's built-in HTTP server functionality to provide communication with the iOS application.

The Python application exposes several HTTP endpoints, with each endpoint responsible for a specific system operation. This structure allows the individual functions of the system to be accessed through clearly defined requests and simplifies testing and debugging during development.

Table 5.1 – HTTP endpoints implemented on the Raspberry Pi

Endpoint	Function
/stream	Provides the live MJPEG camera stream.
/start-recording	Starts recording video on the Raspberry Pi.
/stop-recording	Stops the current recording.
/latest-recording	Returns the filename of the most recent recording.
/download/<filename>	Downloads a recorded video file.
/parameters	Receives machine operating parameters from the iOS application.

The live video stream is generated by the Raspberry Pi Camera Module 3 through Picamera2. Frames are encoded as JPEG images and transmitted continuously using a multipart MJPEG response. For recording, an H.264 encoder and FFmpeg output are used to create MP4 video files on the Raspberry Pi.

The HTTP request-handling mechanism is implemented using Python's `BaseHTTPRequestHandler`. The `do_GET()` method identifies the requested endpoint and executes the corresponding operation. Recording requests initialise or stop the H.264 recording process, while the download-related endpoints locate and transfer previously recorded MP4 files.

A simplified example of the request-handling structure is shown below:

```
def do_GET(self):
    path = urlparse(self.path).path

    if path == "/start-recording":
        ...

    if path == "/stop-recording":
        ...

    if path == "/latest-recording":
        ...

    if path.startswith("/download/"):
        ...

    if path == "/stream":
        ...
```

The use of a single Python application provided a relatively simple implementation suitable for the prototype. However, because the HTTP server was implemented using Python's basic `HTTPServer` class, the server architecture remained lightweight rather than being designed as a fully concurrent production server. This was sufficient for the prototype's local wireless operation while also providing a straightforward basis for future development.

5.2 iOS Application

A native iOS application was developed to provide the primary interface between the operator and the tennis ball machine. The application was implemented using SwiftUI and is responsible for communicating with the Raspberry Pi, displaying the live camera feed and providing controls for the machine's operating functions.

The user interface was designed to minimise the number of interactions required during operation. The main screen provides access to all essential functions, including live video preview, video recording, downloading recorded footage and transmitting machine operating parameters. This allows the operator to monitor and configure the system from a single interface.

Communication with the Raspberry Pi is performed using asynchronous HTTP requests. User actions, such as starting a recording or sending machine parameters, generate the corresponding request to the Raspberry Pi server. Responses returned by the server are then processed by the application to update the system state.

The application also integrates with native iOS services where required. Downloaded recordings are automatically saved to the device Photo Library after the appropriate user permissions have been granted. This removes the need for external file management and allows recorded training sessions to be accessed directly through the Photos application.

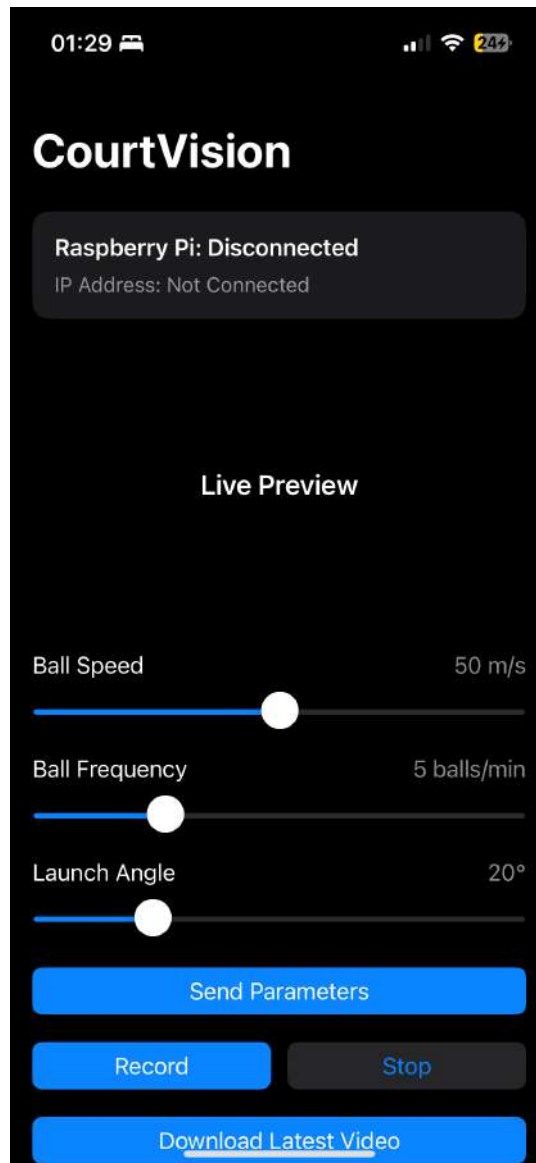


Figure 5.2 shows the main interface of the developed iOS application.

The iOS application was developed using Xcode, with the user interface and application functionality implemented using Swift. Xcode was used to develop, test and organise the application source code, including the live camera preview, recording controls, video download functionality, Raspberry Pi discovery and parameter transmission.

Figure 5.3 shows the Xcode development environment with the CourtVision application source code. The complete set of iOS source files is provided in the accompanying project ZIP folder, allowing the implementation to be examined in greater detail without reproducing the full source code within this report.

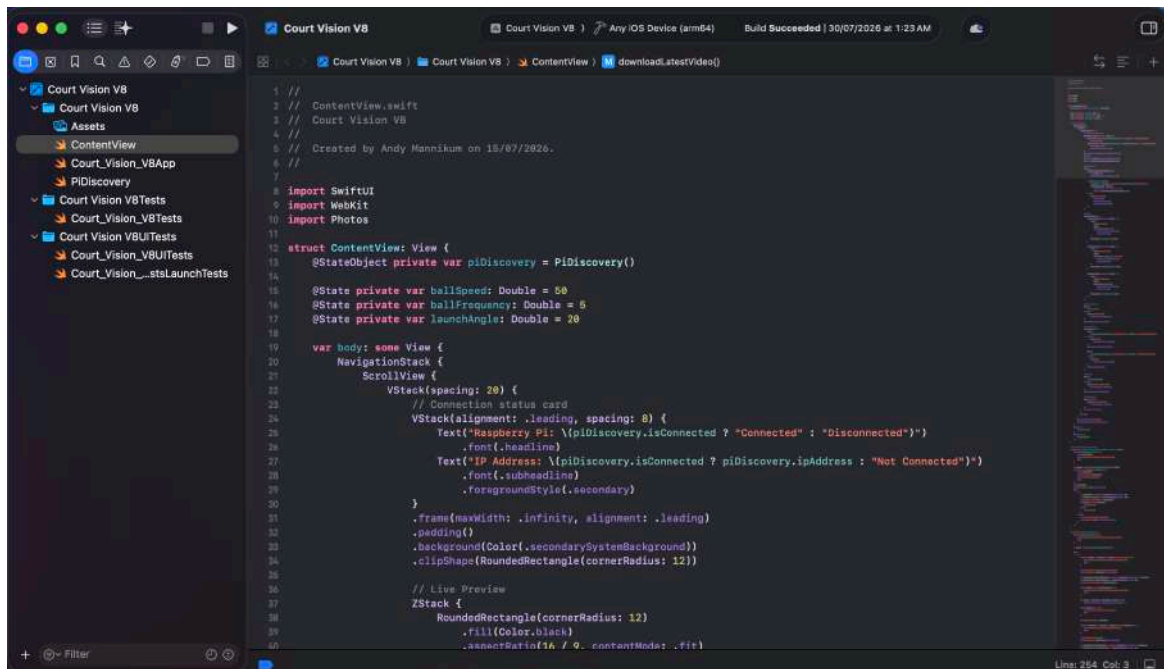


Figure 5.3 – CourtVision iOS application development in Xcode.

5.3 Live Video Streaming

A live video preview was implemented to allow the operator to monitor the tennis ball machine remotely during operation. The video feed is captured using the Raspberry Pi Camera Module 3 and streamed continuously to the iOS application.

The Raspberry Pi generates an MJPEG stream using the Picamera2 library and serves it through the Python HTTP server. The stream is made available through a dedicated HTTP endpoint (`/stream`), allowing the mobile application to request the video feed using the Raspberry Pi's IP address.

On the iOS application, the live preview is displayed using a WebView, which loads the MJPEG stream directly from the Raspberry Pi. This approach provided a straightforward method of displaying continuous video without requiring additional streaming software or complex media frameworks.

The implementation allows the operator to observe the machine in real time while simultaneously accessing the remaining application functions, including recording control and parameter adjustment.

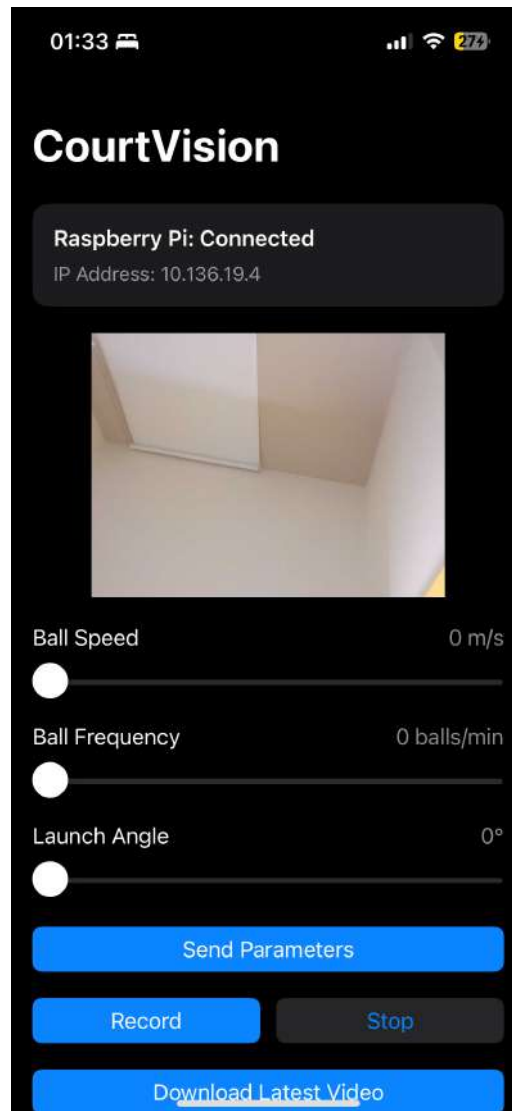


Figure 5.4 shows the live video preview displayed within the iOS application.

A simplified extract of the streaming endpoint is shown below.

```
// Live Preview
```

```
ZStack {
```

```
    RoundedRectangle(cornerRadius: 12)
```

```
        .fill(Color.black)
```

```
        .aspectRatio(16 / 9, contentMode: .fit)
```

```
    if piDiscovery.isConnected,
```

```
        let streamURL = URL(
```

```

        string: "http://(piDiscovery.ipAddress):8080/stream"
    ) {
        StreamPreview(url: streamURL)
            .aspectRatio(16 / 9, contentMode: .fit)
            .clipShape(
                RoundedRectangle(cornerRadius: 12)
            )
    } else {
        Text("Live Preview")
            .font(.title3)
            .fontWeight(.semibold)
            .foregroundColor(.white)
    }
}

```

5.4 Video Recording

Video recording was implemented to allow training sessions to be captured directly on the Raspberry Pi for later analysis. Recording is initiated and terminated from the iOS application, allowing the operator to control the process remotely without interacting with the Raspberry Pi.

When the **Record** button is pressed, the iOS application sends an HTTP request to the Raspberry Pi requesting that recording begins. The Python server processes the request and starts a recording process using the Raspberry Pi Camera Module 3. Video files are stored locally on the Raspberry Pi in MP4 format using a timestamp-based filename to ensure that each recording is uniquely identified.

Selecting the **Stop** button sends a second HTTP request to terminate the recording process. Once recording has stopped, the generated video file remains stored on the Raspberry Pi and becomes available for download through the mobile application.

Separating recording control from video storage allows recordings to be generated without placing additional processing load on the iPhone. It also provides a central location for managing recorded sessions before they are transferred to the mobile device.

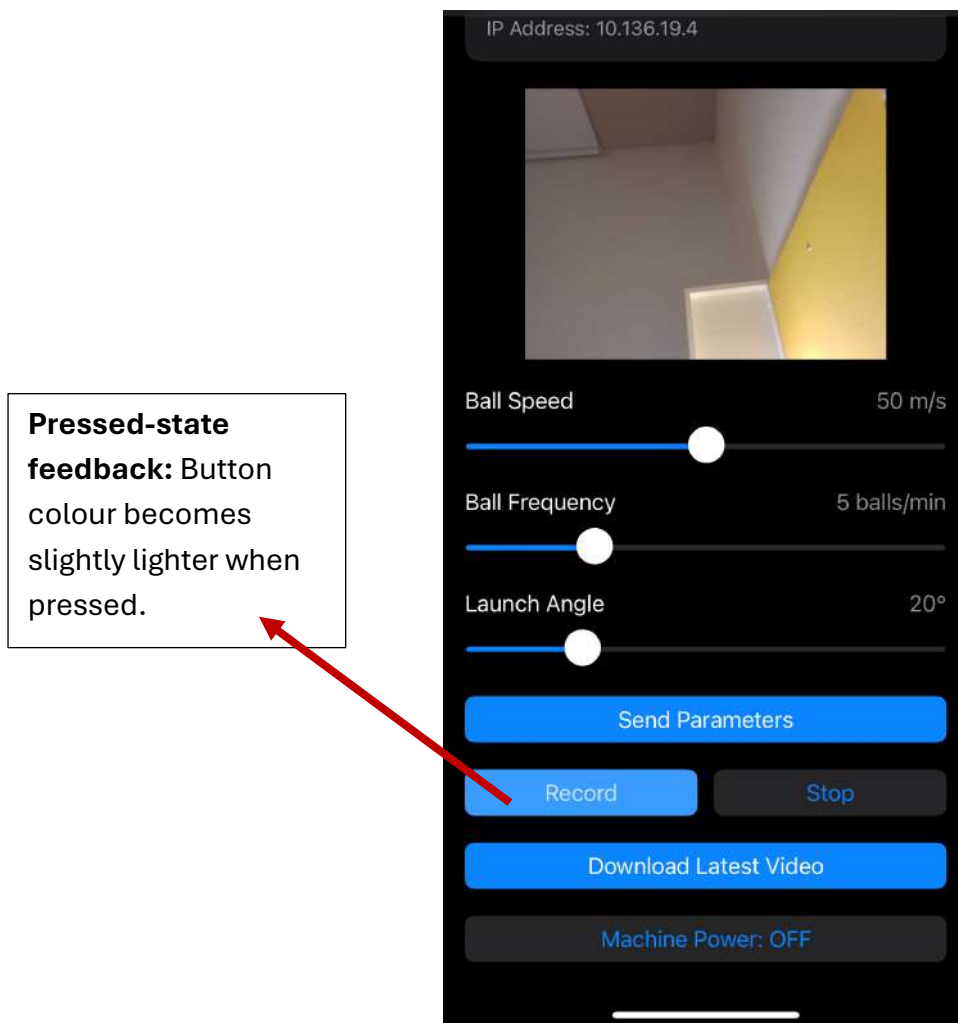


Figure 5.5 – Video recording function implemented in the iOS application.

A simplified extract of the recording request is shown below.

```
// Milestone 5: Record / Stop only

private func sendRecordingRequest(path: String, actionName: String) {

    guard piDiscovery.isConnected else {

        print("\(actionName)")

        print("Failure: Raspberry Pi is not connected")

        return

    }
}
```

```

let urlString = "http://(piDiscovery.ipAddress):8080\(path)"

guard let url = URL(string: urlString) else {

    print("\(actionName)")

    print("Request URL: \(urlString)")

    print("Failure: Invalid URL")

    return

}

print(actionName)

print("Request URL: \(url.absoluteString)")

Task {

    do {

        let (_, response) = try await URLSession.shared.data(from: url)

        let statusCode = (response as? HTTPURLResponse)?.statusCode ?? -1

        print("Response status code: \(statusCode)")

        if (200...299).contains(statusCode) {

            print("Success")

        } else {

            print("Failure")

        }

    } catch {

        print("Response status code: none")

        print("Failure: \(error.localizedDescription)")

    }

}

```

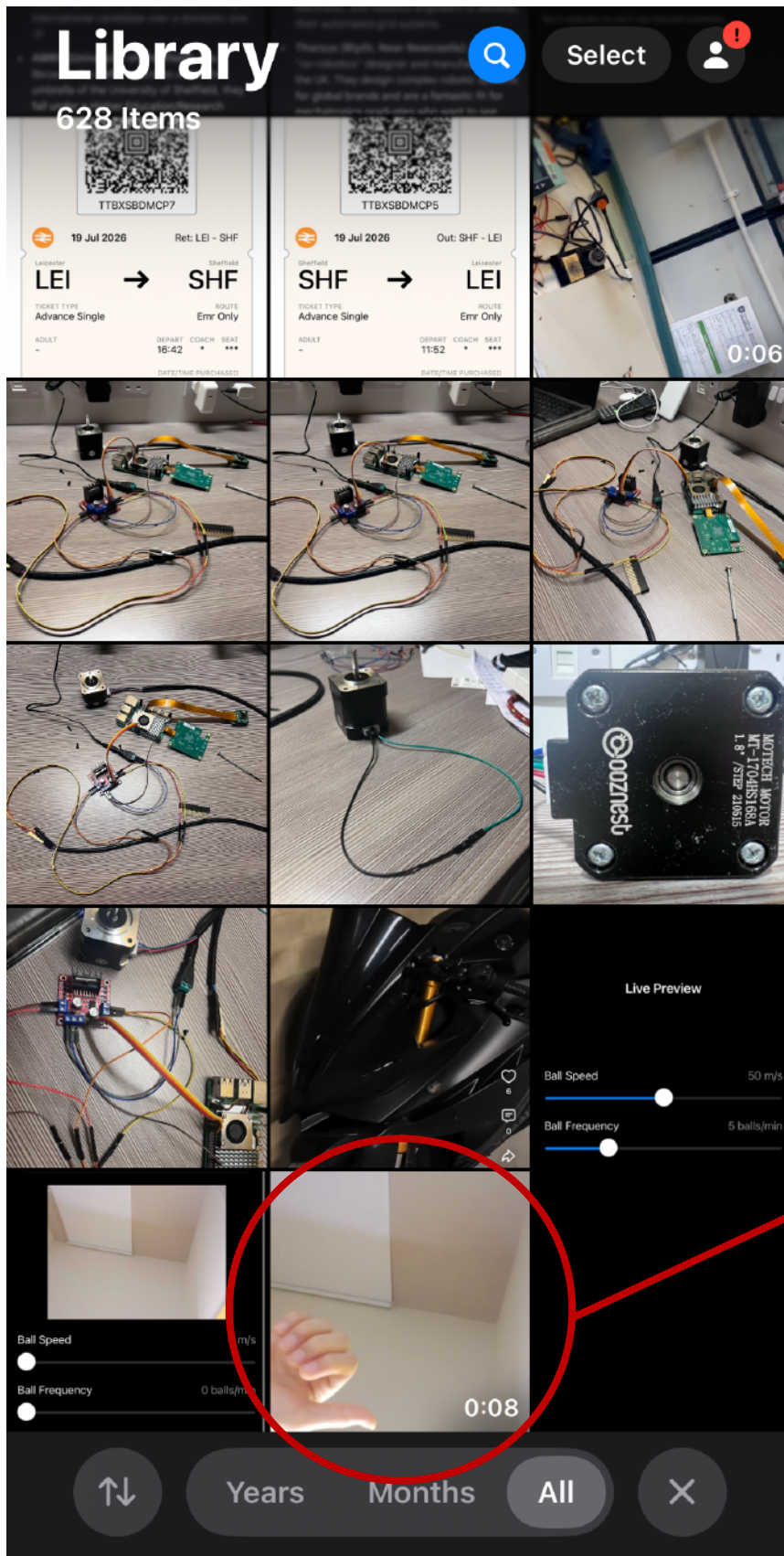
5.5 Video Download

The video download function was implemented to allow recordings stored on the Raspberry Pi to be transferred directly to the operator's iPhone. This enables recorded training sessions to be reviewed or shared without requiring access to the Raspberry Pi file system.

When the **Download Latest Video** button is selected, the iOS application first sends a request to the Raspberry Pi to obtain the filename of the most recently recorded video. After receiving the filename, a second HTTP request is sent to download the corresponding MP4 file.

Once the download has completed, the video is temporarily stored within the application before being saved to the iPhone Photo Library using the native Photos framework. This process requires user permission to access the Photo Library and allows downloaded recordings to appear alongside videos captured directly on the device.

Implementing the download process in two stages simplified file management on the Raspberry Pi by ensuring that only the latest recording was requested by the mobile application. This also reduced user interaction, as the operator is not required to browse or manually select files before downloading.



Downloaded recording:
The recorded video from the Raspberry Pi is successfully downloaded and appears in the Phone gallery.

Figure 5.6 – Downloaded recording stored in the iPhone Photo Library.

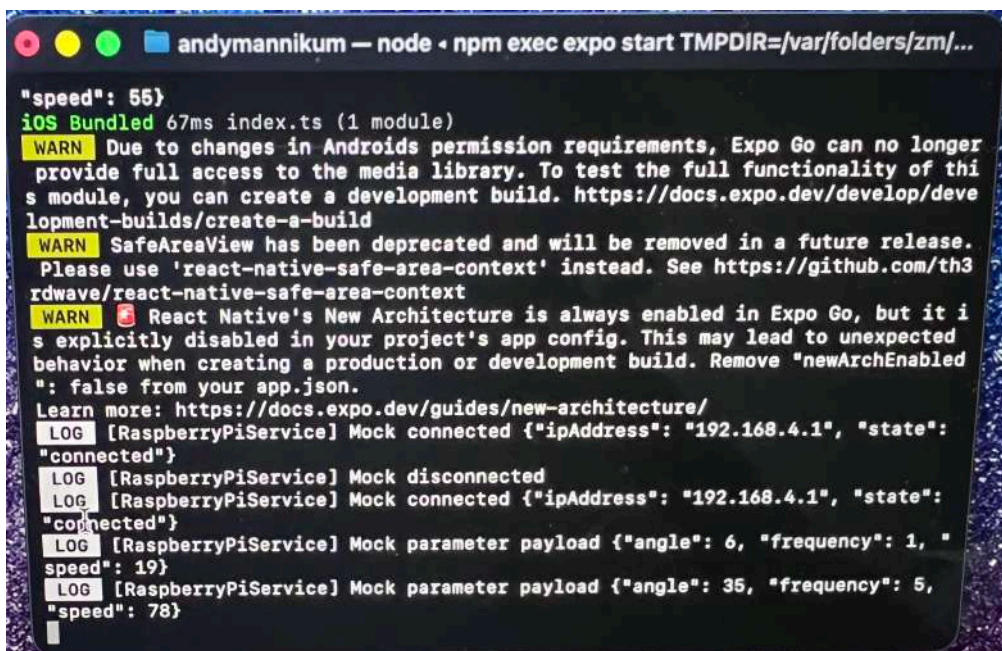
5.6 Parameter Transmission

A parameter transmission function was implemented to allow operating settings to be sent from the iOS application to the Raspberry Pi. This provides a communication interface between the user application and the hardware controller, allowing machine parameters to be modified remotely.

The application allows the user to specify three operating parameters: ball speed, launch angle and ball frequency. When the **Send Parameters** button is pressed, the selected values are packaged into an HTTP request and transmitted to the Raspberry Pi over the local wireless network.

A Python HTTP server running on the Raspberry Pi receives the request, extracts the transmitted values and validates that the parameters have been received correctly. During development, the received values were displayed in the Raspberry Pi terminal to verify successful communication between the mobile application and the server.

Implementing the parameter transmission independently from the motor control simplified software development by allowing communication between the mobile application and Raspberry Pi to be verified before hardware integration. This modular approach reduced debugging complexity and ensured that the communication interface was fully operational before controlling external devices.



```
andymanikun — node • npm exec expo start TMPDIR=/var/folders/zm/...

"speed": 55}
iOS Bundled 67ms index.ts (1 module)
WARN Due to changes in Androids permission requirements, Expo Go can no longer
provide full access to the media library. To test the full functionality of thi
s module, you can create a development build. https://docs.expo.dev/develop/deve
lopment-builds/create-a-build
WARN SafeAreaView has been deprecated and will be removed in a future release.
Please use 'react-native-safe-area-context' instead. See https://github.com/th3
rdwave/react-native-safe-area-context
WARN React Native's New Architecture is always enabled in Expo Go, but it i
s explicitly disabled in your project's app config. This may lead to unexpected
behavior when creating a production or development build. Remove "newArchEnabled
": false from your app.json.
Learn more: https://docs.expo.dev/guides/new-architecture/
LOG [RaspberryPiService] Mock connected {"ipAddress": "192.168.4.1", "state":
"connected"}
LOG [RaspberryPiService] Mock disconnected
LOG [RaspberryPiService] Mock connected {"ipAddress": "192.168.4.1", "state":
"connected"}
LOG [RaspberryPiService] Mock parameter payload {"angle": 6, "frequency": 1, "
speed": 19}
LOG [RaspberryPiService] Mock parameter payload {"angle": 35, "frequency": 5,
"speed": 78}
```

Figure 5.7 shows the corresponding values received by the Raspberry Pi.

During testing, the transmitted parameter values were printed to the Raspberry Pi terminal whenever the **Send Parameters** button was pressed. Two separate sets of parameter values were recorded during testing, demonstrating that different values entered through the iOS application were successfully transmitted to the Raspberry Pi.

Figure 5.7 shows the Raspberry Pi terminal displaying the parameter values received during two transmission tests. The change in the printed values corresponds to the different settings selected in the iOS application, providing direct evidence that the parameter data was being transferred over the wireless connection.

A simplified extract of the parameter transmission process is shown below.

```
private func sendParameters() {  
  
    guard piDiscovery.isConnected else {  
  
        print("Send Parameters failed: Raspberry Pi is not connected")  
  
        return  
  
    }  
  
    let urlString = "http://(piDiscovery.ipAddress):8080/parameters"  
  
    guard let url = URL(string: urlString) else {  
  
        print("Request URL: \(urlString)")  
  
        print("Failure: Invalid URL")  
  
        return  
  
    }  
  
    let body: [String: Double] = [  
  
        "speed": ballSpeed,  
  
        "frequency": ballFrequency,  
  
        "angle": launchAngle  
  
    ]  
  
    guard let jsonData = try? JSONSerialization.data(withJSONObject: body),  
  
        let jsonString = String(data: jsonData, encoding: .utf8) else {  
  
        print("Failure: Could not encode JSON")  
  
        return  
  
    }  
}
```

```

    print("Request URL: \${url.absoluteString}")

    print("JSON body: \${jsonString}")

    var request = URLRequest(url: url)

    request.httpMethod = "POST"

    request.setValue("application/json", forHTTPHeaderField: "Content-Type")

    request.httpBody = jsonData

    Task {

        do {

            let (_, response) = try await URLSession.shared.data(for: request)

            let statusCode = (response as? HTTPURLResponse)?.statusCode ?? -1

            print("Response status code: \${statusCode}")

            if (200...299).contains(statusCode) {

                print("Success")

            } else {

                print("Failure")

            }

        } catch {

            print("Response status code: none")

            print("Failure: \${error.localizedDescription}")

        }

    }

}

```

5.7 Wireless Network Configuration

Wireless connectivity was implemented to allow the iOS application to communicate directly with the Raspberry Pi without relying on an external wireless router. The Raspberry Pi was configured as a local wireless access point using the NetworkManager service available within the Linux operating system.

A dedicated wireless hotspot named **CourtVision-Pi** was configured on the Raspberry Pi. The hotspot was password protected and provided the wireless network used by the iPhone during system operation. Once the hotspot was enabled, it was visible from the iPhone's Wi-Fi settings and could be selected in the same manner as a conventional wireless network.

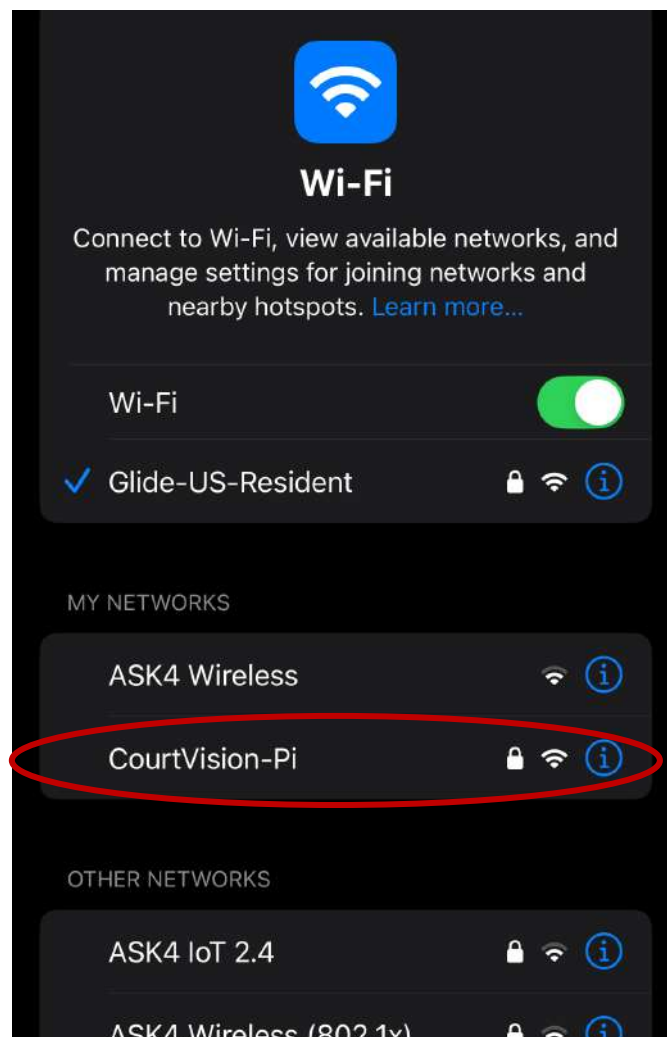
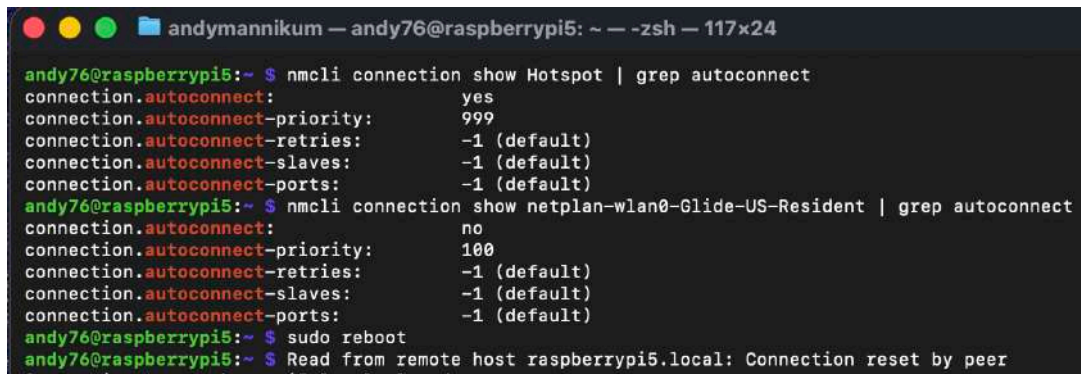


Figure 5.8 shows the “CourtVision-Pi” wireless network detected by the iPhone during testing.

The Raspberry Pi was also configured to prioritise its own hotspot connection during startup. NetworkManager connection priorities were adjusted so that the dedicated **CourtVision-Pi** hotspot had a higher automatic-connection priority than the previously configured external Wi-Fi network. This was intended to ensure that the Raspberry Pi could provide its own local network for subsequent users without requiring manual reconnection to an external network.

The wireless configuration was verified using the Linux `nmcli` command-line utility. The connection configuration could be inspected using commands such as:

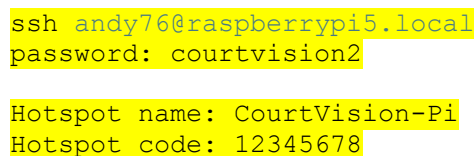


```
andy76@raspberrypi5:~$ nmcli connection show Hotspot | grep autoconnect
connection.autoconnect: yes
connection.autoconnect-priority: 999
connection.autoconnect-retries: -1 (default)
connection.autoconnect-slaves: -1 (default)
connection.autoconnect-ports: -1 (default)
andy76@raspberrypi5:~$ nmcli connection show netplan-wlan0-Glide-US-Resident | grep autoconnect
connection.autoconnect: no
connection.autoconnect-priority: 100
connection.autoconnect-retries: -1 (default)
connection.autoconnect-slaves: -1 (default)
connection.autoconnect-ports: -1 (default)
andy76@raspberrypi5:~$ sudo reboot
andy76@raspberrypi5:~$ Read from remote host raspberrypi5.local: Connection reset by peer
```

The resulting configuration showed that the hotspot was configured for automatic connection with a higher priority than the external Wi-Fi connection. The relevant configuration and verification output are included in Appendix A.

SSH Access During Development

Secure Shell (SSH) was used during development to remotely access the Raspberry Pi from a computer connected to the same local network. This allowed Python programs, camera software and NetworkManager configuration to be controlled without requiring a monitor, keyboard or mouse to remain connected to the Raspberry Pi.



```
ssh andy76@raspberrypi5.local
password: courtvision2

Hotspot name: CourtVision-Pi
Hotspot code: 12345678
```

After entering the appropriate credentials, the Linux terminal of the Raspberry Pi could be accessed remotely. This was particularly useful for configuring the wireless hotspot, running the Python server and testing the camera functionality.

For security reasons, the actual Wi-Fi and SSH passwords are not reproduced in the report. The configuration details required to reproduce the setup are documented separately in the project appendix where appropriate.

5.8 System Integration

Following the implementation of the individual software components, the complete system was integrated to verify that all functions operated correctly as a single platform. Integration testing focused on ensuring reliable communication between the iOS application and the Raspberry Pi while maintaining stable operation of each implemented feature.

The Raspberry Pi was configured to automatically establish its wireless hotspot during startup and execute the camera streaming service. Once connected to the hotspot, the iOS application

automatically established communication with the Raspberry Pi and enabled all implemented functions through a single user interface.

The integrated system was successfully demonstrated with the following functionality operating simultaneously:

- Live camera preview.
- Remote control of video recording.
- Download of recorded video files.
- Transmission of operating parameters.
- Standalone wireless communication through the Raspberry Pi hotspot.

Integration testing confirmed that each software module functioned correctly without affecting the operation of the remaining system components. This modular implementation significantly simplified debugging throughout development, as individual functions could be verified independently before being incorporated into the complete system.

The completed software platform provides the foundation for the final stage of the project, where the transmitted operating parameters will be used to control the speed of the tennis ball machine through the stepper motor.

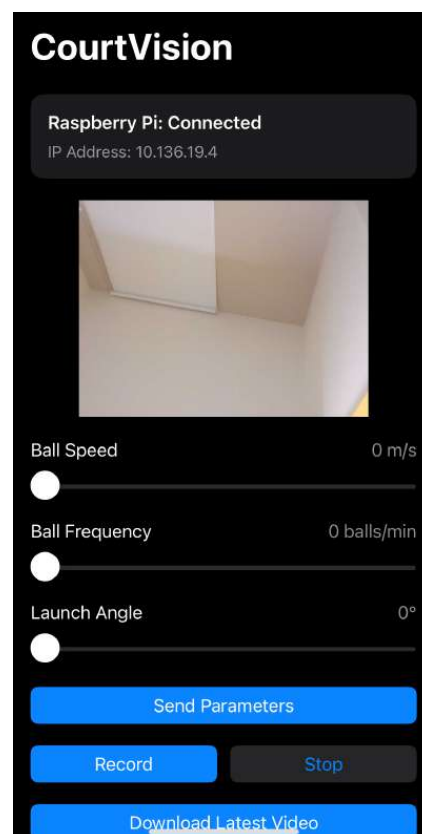


Figure 5.9 – Integrated software system demonstrating live preview, recording, download and parameter transmission

6 Testing and Evaluation

The developed system was tested incrementally throughout the project to verify the functionality of individual components before full system integration. Testing was carried out on the Raspberry Pi 5, Pi Camera Module 3, mobile application, wireless communication interface and the stepper motor control subsystem.

The live camera streaming system was successfully implemented using the Raspberry Pi and Pi Camera Module 3. Video streaming was transmitted over a local Wi-Fi network and displayed correctly within the mobile application. The application maintained a stable connection with the Raspberry Pi and demonstrated low-latency video transmission suitable for monitoring the tennis ball machine remotely.

The mobile application was tested for connectivity, user interface responsiveness and data communication. Mock performance data, including launch speed, launch angle and ball frequency, were successfully transmitted from the Raspberry Pi to the application and displayed correctly. This verified the communication architecture and demonstrated the feasibility of integrating future real-time performance metrics generated by the computer vision system.

The stepper motor control subsystem was also investigated using a Raspberry Pi 5, L298N motor driver and a four-wire stepper motor configuration. Multiple control approaches were evaluated, including custom control software and reference implementations based on standard half-step sequences. During testing, the motor coils were successfully energised, producing holding torque that confirmed communication between the Raspberry Pi and the motor driver. However, reliable rotational motion could not be achieved within the available project timeframe despite extensive software and hardware debugging.

Overall, the project successfully demonstrated the integration of the camera system, wireless communication architecture and mobile application. Although the motor control subsystem requires further investigation, the developed platform provides a functional foundation for future autonomous launcher control and intelligent tennis training applications.

7 Challenges and Limitations

Several challenges were encountered during the development of the smart tennis ball machine, particularly during the integration of the mobile application, Raspberry Pi and motor-control hardware. Although several individual subsystems were successfully developed and tested, integrating the complete system required iterative debugging, practical hardware investigation and changes to the initial development approach.

7.1 Mobile Application Development

The initial mobile application prototype was developed using **Expo Go**, which provided a convenient environment for rapidly testing the user interface and communication with the Raspberry Pi. This approach allowed the basic application functionality and wireless

communication to be explored without requiring a fully native iOS development environment.

As development progressed, however, the application required several forms of HTTP communication to operate together. These included continuous live video streaming, recording-control requests, parameter transmission and video-file transfers. Managing these communication requirements reliably within the initial Expo Go implementation became increasingly difficult. The application was therefore subsequently developed using a native iOS approach, providing greater control over the networking and video-handling behaviour required by the system.

This change also allowed the communication architecture to be separated into individual functions, with the iOS application handling the user interface and network requests while the Raspberry Pi handled the camera, streaming and recording services.

7.2 Stepper Motor Identification and Control

A significant hardware challenge was the integration of the stepper motor with the Raspberry Pi and L298N motor driver. The motor was not supplied with a pin or coil identification sheet, making it initially unclear which motor terminals corresponded to the individual windings.

An experimental identification procedure was therefore used to determine the motor coil pairs. Candidate terminals were temporarily connected and the motor shaft was rotated manually. When terminals belonging to the same winding were connected, the motor exhibited increased resistance to manual rotation due to the electrical interaction within the winding. This provided a practical method of identifying the coil pairs in the absence of manufacturer documentation.





Figure 7.2 shows the experimental procedure used during the motor-winding identification process.

Following identification of the coil pairs, the motor was connected to the L298N driver and several GPIO configurations and stepping sequences were investigated. Although the Raspberry Pi was able to generate control signals and the motor driver was able to energise the motor coils, consistent rotational motion could not be achieved.

Multiple software implementations, GPIO configurations and stepping sequences were therefore tested. However, reliable automated rotation remained unresolved within the available project timeframe. Consequently, the launcher positioning mechanism could not be integrated into the final mobile application.

7.3 Limited Hardware Diagnostic Equipment

Another limitation was the availability of hardware diagnostic equipment. Without instruments such as a multimeter or oscilloscope, it was difficult to directly verify electrical signals, measure current delivery or isolate potential faults within the motor driver and stepper-motor circuit.

As a result, troubleshooting of the motor subsystem relied primarily on software testing, GPIO configuration changes, manual observation and the physical behaviour of the motor. This limited the ability to conclusively determine whether the remaining issue was associated with the motor, driver, wiring, power delivery or control sequence.

7.4 Project Time Constraints

The project was also constrained by the available internship timeframe. Development priorities were therefore focused on implementing and stabilising the core functionality of the system, including Raspberry Pi integration, wireless camera streaming, video recording and the iOS mobile application.

This prioritisation allowed a functional software and camera platform to be established while the more complex motor-control subsystem remained under investigation. The resulting system therefore provides a working foundation for further development of the mechanical launcher and automated positioning mechanism.

Despite these limitations, the completed work demonstrates the feasibility of integrating embedded hardware, wireless networking, camera streaming and mobile software within a smart sports-training platform. The principal unresolved issue was associated with the hardware implementation of the motor-control subsystem rather than the underlying mobile-to-Raspberry-Pi communication architecture.

8 Future Improvements

Although the project successfully established the core communication framework between the Raspberry Pi, mobile application and camera system, several enhancements could be implemented to further improve the functionality and performance of the smart tennis ball machine.

The highest priority for future work is the implementation of reliable launcher positioning through stepper motor control. Replacing the L298N motor driver with a dedicated stepper motor driver, such as the A4988 or DRV8825, would provide improved current regulation and more accurate motor control. This would enable precise adjustment of the launcher angle directly from the mobile application.

The next stage of development would involve integrating the TrackNet computer vision model into the Raspberry Pi. This would allow automatic detection and tracking of tennis balls during training sessions, enabling real-time analysis of ball trajectories and shot placement. The processed data could then be transmitted to the mobile application to provide performance statistics and visual feedback to the user.

The mobile application could also be expanded by introducing additional user features such as launcher calibration, configurable training programmes, shot history and data logging. Real-time control of the launcher speed, launch frequency and launch angle could be incorporated to provide a more comprehensive training experience.

Finally, further optimisation of the system architecture could improve overall performance. Techniques such as multithreaded processing, hardware acceleration using the Raspberry Pi AI HAT and more efficient wireless communication protocols would reduce latency and improve responsiveness during real-time operation. These improvements would transform the

current prototype into a more capable and intelligent sports training platform suitable for future research and commercial development.

9 Conclusion

This project successfully demonstrated the development of a smart tennis ball machine control system by integrating embedded hardware, wireless networking and mobile application technologies. The primary objective was to establish communication between a Raspberry Pi 5, Pi Camera Module 3 and a mobile application, providing a foundation for future intelligent tennis training systems.

A live camera streaming system was successfully implemented, allowing real-time video captured by the Raspberry Pi camera to be viewed within the mobile application over a local Wi-Fi network. Communication between the Raspberry Pi and the application was also established, enabling the transmission and display of launcher parameters and mock performance data. These achievements demonstrate the feasibility of using a Raspberry Pi as the central processing unit for an intelligent sports training platform.

The project also investigated the implementation of launcher positioning using a stepper motor controlled through an L298N motor driver. Although communication between the Raspberry Pi and the motor driver was successfully established and the motor coils were energised, reliable rotational motion could not be achieved within the available project timeframe. Extensive software and hardware testing was carried out, and the work completed provides a useful basis for future investigation using dedicated stepper motor drivers and additional diagnostic equipment.

Overall, the project successfully established the core software, communication and camera infrastructure required for the proposed smart tennis ball machine, while the motor-control subsystem remained an area for further development.